Dev Ops Project- 1 Assignment of Mind Track Application Deployment

Deploy the given React application to a production ready state

Prepared by: Md Rustam Ansari *Hcl Gui Batch:* DO-C-WE-E-B36

Project Tasks contents & Timeline:

Task	Descriptions	Page	Duration
1.	Project Tasks Contents, Project Summary & Project Scope	1-1	-- --
2.	Clone the Application & Set Up Git Repository	2-3	-- --
3.	Dockerize the Application & (Containerization)	4-6	-- --
4.	Set Up AWS Elastic Container Registry (ECR)	6-7	-- --
5.	Set Up Amazon EKS Cluster	6-6	-- --
6.	Write Kubernetes Manifests	7-7	-- --
7.	AWS CodeBuild for Project	8-8	-- --
8.	Push Code to GitHub Repository	9-9	-- --
9.	Configure IAM Roles & CodeBuild Project	10-10	-- --
10.	Configure AWS CodePipeline	11-11	-- --
11.	Verify, Monitor, and Gather Deliverables	12-12	-- --

Project Summary URL & Project Scope: https://docs.google.com/document/d/1ZwIfa22Li657eWY-AVVI7R4jzqvODsScLAJfn7fXX_g

» *Tech stacks need to be used:*
- Docker
- AWS
- DevOps

» *How do I submit my work?*
- Push all your work files to GitHub (O/P screenshot images must).
- Submit your URLs in the portal.

» *Terms and Conditions?*
- You agree to not share this confidential document with anyone.
- You agree to open-source your code (it may even look good on your profile!). Do not mention our company's name anywhere in the code.
- We will never use your source code under any circumstances for any commercial purposes; this is just a basic assessment task.

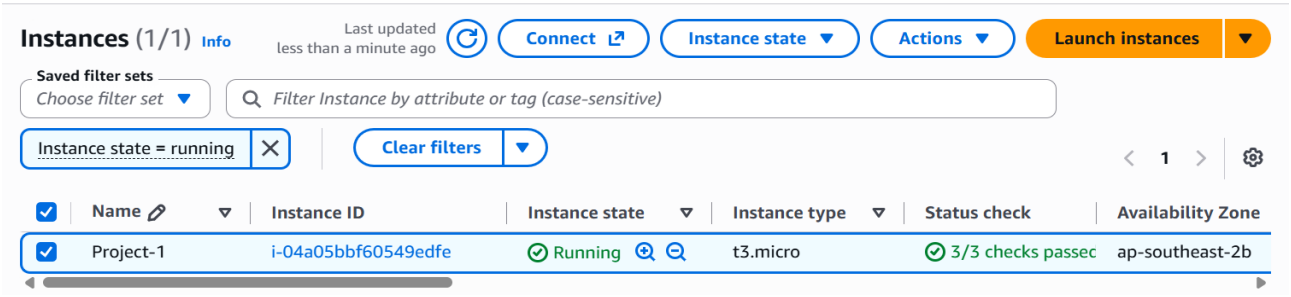
NOTE: Any violation of Terms and conditions is strictly prohibited. You are bound to adhere to it.

Clone the Application & Set Up Git Repository:

» Clone the below mentioned repository and deploy the application (Run application in port 3000).
- *Repo URL:* https://github.com/Vennilavanguvi/Brain-Tasks-App.git

Clone the Application & Set Up Git Repository Procedures (workflow):

» Project_Steps_1: Clone Repo for Application Deployment
- Deploy an ec2 instance, access them using ssh (22 port), Install prerequisites and the clone repo.



- Access the instance post successfully launched.

```
md_rustam@DESKTOP-CPK0PUB:~$ chmod 400 keypair.pem
md_rustam@DESKTOP-CPK0PUB:~$ ssh -i "keypair.pem" ubuntu@ec2-16-176-101-64.ap-southeast-2.compute.amazonaws.com
Welcome to Ubuntu 26.04 LTS (GNU/Linux 7.0.0-1006-aws x86_64)

 * Documentation:  https://docs.ubuntu.com
 * Management:     https://landscape.canonical.com
 * Support:        https://ubuntu.com/pro

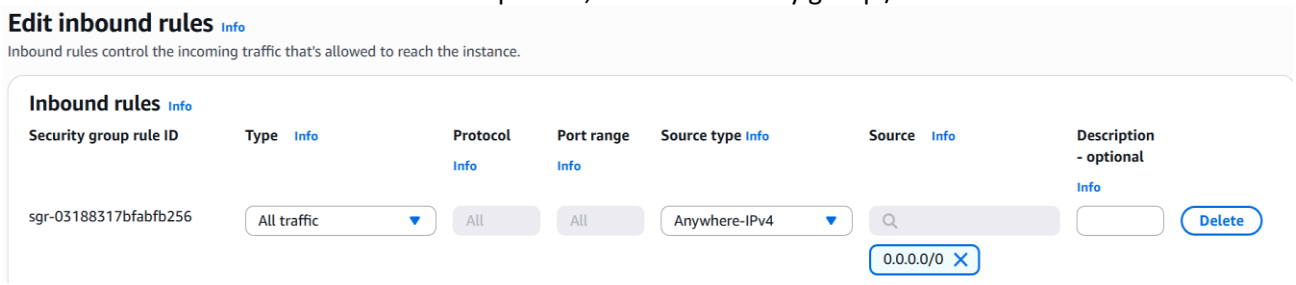
 System information as of Sat Oct 17 07:22:09 UTC 2026
```

- Update the latest OS patch, install prerequisite packages

```
ubuntu@ip-172-31-33-28:~$ sudo apt update -y; sudo apt upgrade -y
Hit:1 http://ap-southeast-2.ec2.archive.ubuntu.com/ubuntu resolute InRelease
Hit:2 http://ap-southeast-2.ec2.archive.ubuntu.com/ubuntu resolute-updates InRelease
Hit:3 http://ap-southeast-2.ec2.archive.ubuntu.com/ubuntu resolute-backports InRelease
Hit:4 http://security.ubuntu.com/ubuntu resolute-security InRelease
All packages are up to date.
The following package was automatically installed and is no longer required:
  pollinate
Use 'sudo apt autoremove' to remove it.

Summary:
  Upgrading: 0, Installing: 0, Removing: 0, Not Upgrading: 0
```

- Allowed inbound all traffic instead of “port 22, 3000” in Security group / firewall.



- Create a directory “Project-1_MindTrack” into the ec2 instance and make git clone.

$ git clone https://github.com/Vennilavanguvi/Brain-Tasks-App.git

```
ubuntu@ip-172-31-33-28:~$ mkdir Project-1_MindTrack
ubuntu@ip-172-31-33-28:~$ cd Project-1_MindTrack/
ubuntu@ip-172-31-33-28:~/Project-1_MindTrack$ git clone https://github.com/Vennilavanguvi/Brain-Tasks-App.git
Cloning into 'Brain-Tasks-App'...
remote: Enumerating objects: 11, done.
remote: Counting objects: 100% (2/2), done.
remote: Compressing objects: 100% (2/2), done.
remote: Total 11 (delta 0), reused 0 (delta 0), pack-reused 9 (from 1)
Receiving objects: 100% (11/11), 101.89 KiB | 8.49 MiB/s, done.
```

- Navigate to the cloned Git repository directory and list the contents of the directory and start the serve

```
ubuntu@ip-172-31-33-28:~$ cd Project-1_MindTrack/Brain-Tasks-App/dist/
ubuntu@ip-172-31-33-28:~/Project-1_MindTrack/Brain-Tasks-App/dist$ ls -l
total 16
drwxrwxr-x 2 ubuntu ubuntu 4096 Oct 17 10:46 assets
-rw-rw-r-- 1 ubuntu ubuntu  603 Oct 17 10:46 index.html
drwx------ 3 ubuntu ubuntu 4096 Oct 17 10:52 snap
-rw-rw-r-- 1 ubuntu ubuntu 1497 Oct 17 10:46 vite.svg
```

$ Run server & verify the URL to access them (http://172.31.33.28:3000), (http://16.176.101.64:3000)

```
ubuntu@ip-172-31-33-28:~/Project-1_MindTrack/Brain-Tasks-App/dist$ serve -s . -l 3000
ERROR  Cannot copy server address to clipboard: Both xsel and fallback failed.

   Serving!

   - Local:    http://localhost:3000
   - Network:  http://172.31.33.28:3000

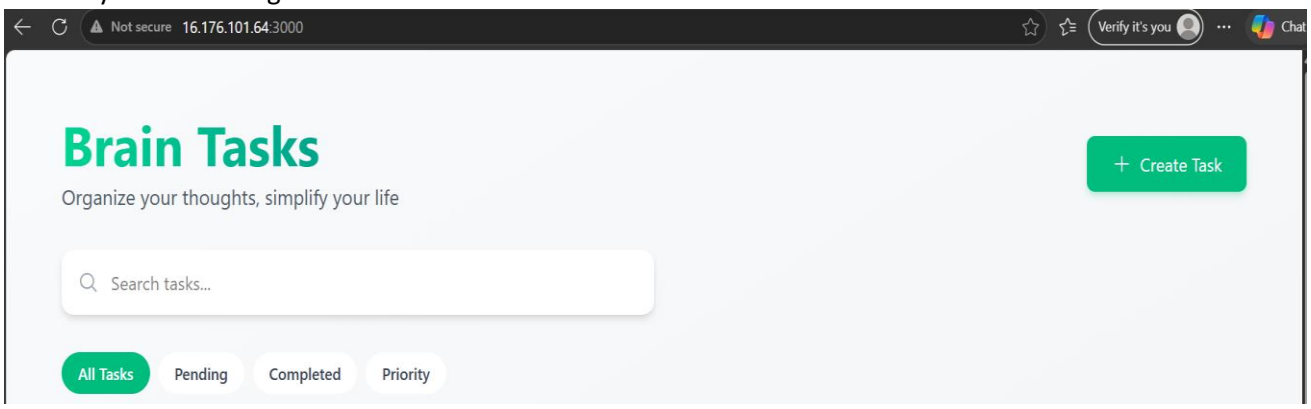
HTTP  10/17/2026 11:13:52 AM 172.31.33.28 GET /
HTTP  10/17/2026 11:13:52 AM 172.31.33.28 Returned 200 in 23 ms
```

- Verify access through the CLI

$ curl htt://172.31.33.28:3000

```
ubuntu@ip-172-31-33-28:~$ curl http://172.31.33.28:3000
<!doctype html>
<html lang="en">
  <head>
    <meta charset="UTF-8" />
    <link rel="icon" type="image/svg+xml" href="https://is1-ssl.mzstatic.com/image/thumb/Purple221/v4/1b/a6/70/1ba67006-
9749-9567-151b-731d5cba4f93/AppIcon-0-0-1x_U007epad-0-1-0-85-220.jpeg/512x512bb.jpg" />
    <meta name="viewport" content="width=device-width, initial-scale=1.0" />
    <title>Brain Task</title>
    <script type="module" crossorigin src="/assets/index-BHGiHu50.js"></script>
    <link rel="stylesheet" crossorigin href="/assets/index-DPTLVrPB.css">
  </head>
  <body>
    <div id="root"></div>
  </body>
</html>
```

- Verify access through the browser



Dockerize the Application & (Containerization):

» Project_Steps_2 (Docker Containerization)

- Dockerize the application by creating Dockerfile
- Build an application and check output using docker image.

Docker_Containarization Application (workflow):

» Project_Steps_2: Create ec2 instance and access them with 22 port to perform this project

- Create Dockerfile for Dockerize application

$ ssh -i "keypair.pem" ubuntu@ec2-16-176-101-64.ap-southeast-2.compute.amazonaws.com

```
md_rustam@DESKTOP-CPK0PUB:~$ ssh -i "keypair.pem" ubuntu@ec2-16-176-101-64.ap-southeast-2.compute.amazonaws.com
Welcome to Ubuntu 26.04.1 LTS (GNU/Linux 7.0.0-1006-aws x86_64)

 * Documentation:  https://docs.ubuntu.com
 * Management:     https://landscape.canonical.com
 * Support:        https://ubuntu.com/pro

 System information as of Sat Oct 17 11:39:26 UTC 2026
```

- Update the latest OS patch, install prerequisite packages

$ sudo apt update -y; sudo apt upgrade -y

```
ubuntu@ip-172-31-33-28:~$ sudo apt update -y; sudo apt upgrade -y
Hit:1 http://ap-southeast-2.ec2.archive.ubuntu.com/ubuntu resolute InRelease
Hit:2 http://ap-southeast-2.ec2.archive.ubuntu.com/ubuntu resolute-updates InRelease
Hit:3 http://ap-southeast-2.ec2.archive.ubuntu.com/ubuntu resolute-backports InRelease
Hit:4 http://security.ubuntu.com/ubuntu resolute-security InRelease
All packages are up to date.
Get more security updates through Ubuntu Pro with 'esm-apps' enabled:
  node-lodash-packages node-shell-quote node-lodash
Learn more about Ubuntu Pro on AWS at https://ubuntu.com/aws/pro
Summary:
  Upgrading: 0, Installing: 0, Removing: 0, Not Upgrading: 0
```

- Create a Dockerfile "In the root of your project (Brain-Tasks-App), using below code.

$ cat Dockerfile

```
ubuntu@ip-172-31-33-28:~/Project-1_MindTrack/Brain-Tasks-App$ cat Dockerfile
# Use official Nginx image
FROM docker.io/library/nginx:alpine

# Set working directory to Nginx's default HTML folder
WORKDIR /usr/share/nginx/html

# Copy built frontend files into container
COPY dist/ .

# Expose port 3000
EXPOSE 3000

# Run Nginx in foreground
CMD ["nginx", "-g", "daemon off;"]
```

- Run commands for Build and run docker container

$ docker build -t dockerapp: latest .

```
ubuntu@ip-172-31-33-28:~/Project-1_MindTrack/Brain-Tasks-App$ docker build -t dockerapp:latest
Command 'docker' not found, but can be installed with:
sudo apt install docker.io      # version 29.1.3-0ubuntu4.1, or
sudo apt install podman-docker  # version 5.7.0+ds2-3build1
```

- Install the required packages as suggested

$ sudo apt install docker.io

```
ubuntu@ip-172-31-33-28:~/Project-1_MindTrack/Brain-Tasks-App$ sudo apt install docker.io
docker.io is already the newest version (29.1.3-0ubuntu4.1).
Summary:
  Upgrading: 0, Installing: 0, Removing: 0, Not Upgrading: 0
```

- Build the docker image using the Dockerfile

$ docker build -t myapp:latest .

```
ubuntu@ip-172-31-33-28:~/Project-1_MindTrack/Brain-Tasks-App$ docker build -t myapp:latest .
[+] Building 6.4s (8/8) FINISHED                                                                                  docker:default
 => [internal] load build definition from Dockerfile                                                                        0.1s
 => => transferring dockerfile: 339B                                                                                        0.0s
 => [internal] load metadata for docker.io/library/nginx:alpine                                                             2.2s
 => [internal] load .dockerignore                                                                                           0.0s
 => => transferring context: 2B                                                                                             0.0s
 => [1/3] FROM docker.io/library/nginx:alpine@sha256:892bb2152464f0c5d8972593a5f5afaf8054552580b699d95f8ff4af08c2           2.2s
 => => resolve docker.io/library/nginx:alpine@sha256:892bb2152464f0c5d8972593a5f5afaf8054552580b699d95f8ff4af08c2           0.0s
 => => sha256:1fdd0614a47ec2896aea36fb280a7b5e8655614dfb56ec1a64002131791de601 20.53MB / 20.53MB                            0.5s
 => => sha256:70683d58fa5c20b5b31c9998a16f6634720d3393dc62934a608754abe449fe3b 1.21kB / 1.21kB                              0.4s
 => => sha256:6638b436592c96dd194db76a6b786b60b344d80ebe160e646c871d4e7eff25b5 1.40kB / 1.40kB                              0.6s
 => => sha256:710ec83ca6cd71fa5636d861343b5660636728f71992fd1b68914e03c7af3ad4 405B / 405B                                  0.6s
 => => sha256:b44956913298a55652fa97af34d296339c089250911e46e1a1c0eab8a88538c7 966B / 966B                                  0.2s
 => => sha256:3bf3671a4e6a18510ca49e3a4a226c2ef734a646e01e34e1ad0eff5faa01e3a0 629B / 629B                                  0.2s
 => => sha256:6711e0c4f5a79e8db637a3412394f29da647e54476d67c5b211a11527568ddc5 4.42MB / 4.42MB                              0.3s
 => => sha256:e2de96513ba9eb53b431787ec8a65cdde380ac4772a3e4c4b714dcfde2a102b5 3.85MB / 3.85MB                              0.3s
 => => extracting sha256:e2de96513ba9eb53b431787ec8a65cdde380ac4772a3e4c4b714dcfde2a102b5                                   0.2s
 => => extracting sha256:6711e0c4f5a79e8db637a3412394f29da647e54476d67c5b211a11527568ddc5                                   0.2s
 => => extracting sha256:3bf3671a4e6a18510ca49e3a4a226c2ef734a646e01e34e1ad0eff5faa01e3a0                                   0.0s
 => => extracting sha256:b44956913298a55652fa97af34d296339c089250911e46e1a1c0eab8a88538c7                                   0.0s
 => => extracting sha256:710ec83ca6cd71fa5636d861343b5660636728f71992fd1b68914e03c7af3ad4                                   0.0s
 => => extracting sha256:70683d58fa5c20b5b31c9998a16f6634720d3393dc62934a608754abe449fe3b                                   0.0s
 => => extracting sha256:6638b436592c96dd194db76a6b786b60b344d80ebe160e646c871d4e7eff25b5                                   0.0s
 => => extracting sha256:1fdd0614a47ec2896aea36fb280a7b5e8655614dfb56ec1a64002131791de601                                   0.6s
 => [internal] load build context                                                                                           0.1s
 => => transferring context: 318.17kB                                                                                       0.0s
 => [2/3] WORKDIR /usr/share/nginx/html                                                                                     0.1s
 => [3/3] COPY dist/ .                                                                                                      0.1s
 => exporting to image                                                                                                      1.4s
 => => exporting layers                                                                                                     0.2s
 => => exporting manifest sha256:d4e74e51318a007a7c88ae7eb6e91d895db7d875073dd83129cbc836d0a3bb4e                           0.0s
 => => exporting config sha256:d8e535fb5ca747114a58f492ed8ff44d52c466f1311ef09fd0dc61930553f639                             0.0s
 => => exporting attestation manifest sha256:7fb8e873ec653902d5c70b6cf152dd5dca447fbb011461ec2ee83c77a7b31320               0.0s
 => => exporting manifest list sha256:af2b488992b23a4394ad2034da9db826aefcd74ec447818ffbc40edededfed791                     0.0s
 => => naming to docker.io/library/myapp:latest                                                                             0.0s
 => => unpacking to docker.io/library/myapp:latest                                                                          1.1s
```

- Verify the built docker image

```
ubuntu@ip-172-31-33-28:~/Project-1_MindTrack/Brain-Tasks-App$ docker images

IMAGE           ID             DISK USAGE   CONTENT SIZE   EXTRA
myapp:latest    af2b488992b2        103MB         28.9MB
```

- Run the docker image and verify the application accessible.

$ docker run -d -p 3000:80 myapp:latest

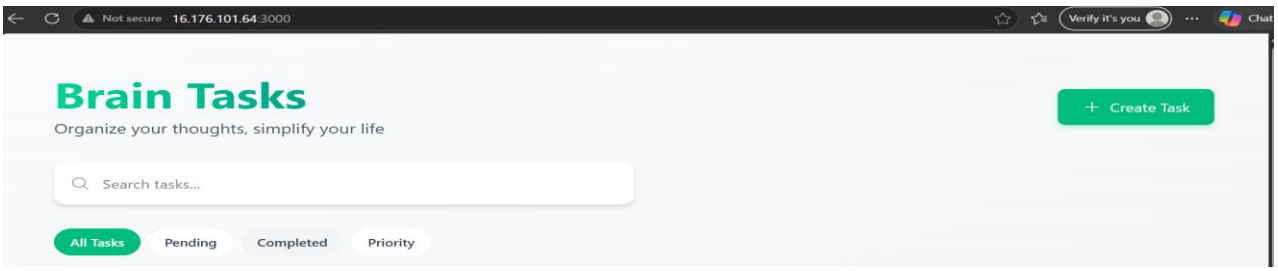
```
ubuntu@ip-172-31-33-28:~/Project-1_MindTrack/Brain-Tasks-App$ docker run -d -p 3000:80 myapp:latest
da5e2a01eb3eb7798ff9cd43a23e1de4a1319cc65b30721dd7e07325b0c04333
ubuntu@ip-172-31-33-28:~/Project-1_MindTrack/Brain-Tasks-App$ docker ps
CONTAINER ID   IMAGE          COMMAND                  CREATED         STATUS         PORTS                                                  NAMES
da5e2a01eb3e   myapp:latest   "/docker-entrypoint.…"   9 seconds ago   Up 8 seconds   3000/tcp, 0.0.0.0:3000->80/tcp, [::]:3000->80/tcp   mystifying_nightingale
```

- Verify the application using CLI

$ curl http://localhost:3000

```
ubuntu@ip-172-31-33-28:~/Project-1_MindTrack/Brain-Tasks-App$ curl http://localhost:3000
<!doctype html>
<html lang="en">
  <head>
    <meta charset="UTF-8" />
    <link rel="icon" type="image/svg+xml" href="https://is1-ssl.mzstatic.com/image/thumb/Purple221/v4/1b/a6/70/1ba67006-9749-9567-151b-731d5cba4f93/AppIcon-0-0-1x_U007epad-
0-1-0-85-220.jpeg/512x512bb.jpg" />
    <meta name="viewport" content="width=device-width, initial-scale=1.0" />
    <title>Brain Task</title>
    <script type="module" crossorigin src="/assets/index-BHGiHu50.js"></script>
    <link rel="stylesheet" crossorigin href="/assets/index-DPTLVrPB.css">
  </head>
  <body>
    <div id="root"></div>
  </body>
</html>
```

- Verify the application using browser



Set Up AWS Elastic Container Registry (ECR) and Docker hub Repository

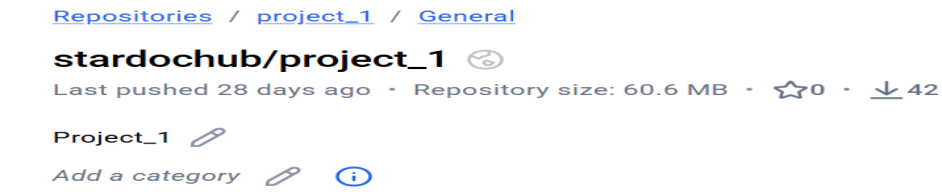
» Registry (ECR and DockerHub Repository):

- Create an AWS ECR or Dockerhub repository for storing docker images.

Steps to Create Docker Hub Repository:

» Project_Steps_3:

- Go to **hub.docker.com**, log in and Click **Create Repository**



- Authenticate the Docker CLI on amazon ec2 instance terminal

$ docker login -u stardochub

```
ubuntu@ip-172-31-33-28:~/Project-1_MindTrack/Brain-Tasks-App$ docker login -u stardochub

i Info → A Personal Access Token (PAT) can be used instead.
         To create a PAT, visit https://app.docker.com/settings

Password:

WARNING! Your credentials are stored unencrypted in '/home/ubuntu/.docker/config.json'.
Configure a credential helper to remove this warning. See
https://docs.docker.com/go/credential-store/

Login Succeeded
```

- Tag and Push the Local Image to docker hub repository

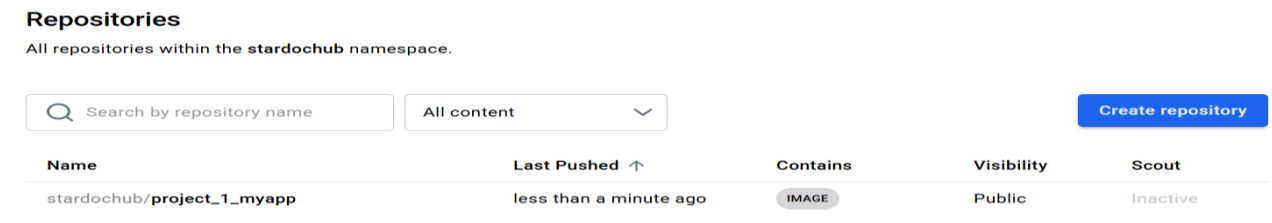
$ docker tag myapp:latest **stardochub/project_1_myapp**:latest

```
ubuntu@ip-172-31-33-28:~/Project-1_MindTrack/Brain-Tasks-App$ docker images
                                                                                    i Info →  U  In Use
IMAGE          ID             DISK USAGE   CONTENT SIZE   EXTRA
myapp:latest   af2b488992b2        103MB         28.9MB
ubuntu@ip-172-31-33-28:~/Project-1_MindTrack/Brain-Tasks-App$ docker tag myapp:latest stardochub/project_1_myapp:latest
ubuntu@ip-172-31-33-28:~/Project-1_MindTrack/Brain-Tasks-App$ docker images
                                                                                    i Info →  U  In Use
IMAGE                                ID             DISK USAGE   CONTENT SIZE   EXTRA
myapp:latest                         af2b488992b2        103MB         28.9MB
stardochub/project_1_myapp:latest    af2b488992b2        103MB         28.9MB
```

- Push the docker image to docker hub repository

```
ubuntu@ip-172-31-33-28:~/Project-1_MindTrack/Brain-Tasks-App$ docker push stardochub/project_1_myapp:latest
The push refers to repository [docker.io/stardochub/project_1_myapp]
6711e0c4f5a7: Mounted from stardochub/project_1
44136fa355b3: Mounted from stardochub/project_1
f8362b33e9ae: Mounted from stardochub/project_1
e2de96513ba9: Mounted from stardochub/project_1
3bf3671a4e6a: Mounted from stardochub/project_1
b44956913298: Mounted from stardochub/project_1
710ec83ca6cd: Mounted from stardochub/project_1
6638b436592c: Mounted from stardochub/project_1
70683d58fa5c: Mounted from stardochub/project_1
1fdd0614a47e: Mounted from stardochub/project_1
4f4fb700ef54: Mounted from stardochub/project_1
ea1652ad0ceb: Mounted from stardochub/project_1
latest: digest: sha256:af2b488992b23a4394ad2034da9db826aefcd74ec447818ffbc40ededefed791 size: 856
```

- Verify latest pushed maige on docker repository using browser



Steps to Create AWS Elastic Container Registry (ECR) Repository:

» Project_Steps_4:

- Ensure you have the AWS CLI installed and configured with appropriate permissions (AmazonEC2ContainerRegistryFullAccess or equivalent IAM policy):
- Install AWS CLI

$ aws configure list

```
ubuntu@ip-172-31-33-28:~/Project-1_MindTrack/Brain-Tasks-App$ aws configure list
NAME        : VALUE                    : TYPE                    : LOCATION
profile     : <not set>                : None                    : None
access_key  : ****************E54D     : shared-credentials-file :
secret_key  : ****************LDL7     : shared-credentials-file :
region      : us-east-1                : config-file             : ~/.aws/config
```

- Check already create docker images available on instance

$ docker images

```
ubuntu@ip-172-31-33-28:~/Project-1_MindTrack/Brain-Tasks-App$ docker images
IMAGE                                ID             DISK USAGE   CONTENT SIZE   EXTRA
myapp:latest                         af2b488992b2       103MB         28.9MB
stardochub/project_1_myapp:latest    af2b488992b2       103MB         28.9MB
```

- Tagging the image to push ECR repository

$ aws ecr-public get-login-password --region us-east-1 | docker login --username AWS --password-stdin public.ecr.aws/t1t8u9r7

```
ubuntu@ip-172-31-33-28:~$ docker tag myapp:latest public.ecr.aws/t1t8u9r7/project-repo:latest
```

- Verify the image post tagging

$ docker images

```
ubuntu@ip-172-31-33-28:~$ docker images
IMAGE                                          ID             DISK USAGE   CONTENT SIZE   EXTRA
myapp:latest                                   af2b488992b2       103MB         28.9MB
public.ecr.aws/t1t8u9r7/project-repo:latest    af2b488992b2       103MB         28.9MB
```

- Login AWS console and create AWS Elastic Container Registry (ECR) Public Repository



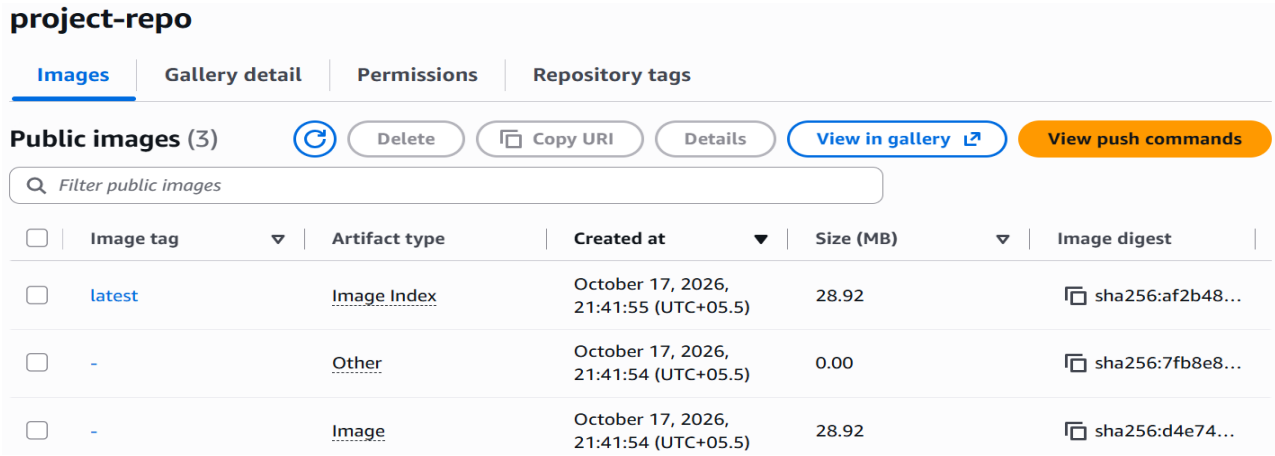
- Retrieve an authentication token and authenticate your Docker client to your registry. Use the AWS CLI

$ aws ecr-public get-login-password --region us-east-1 | docker login --username AWS --password-stdin public.ecr.aws/t1t8u9r7

$ docker push public.ecr.aws/t1t8u9r7/project-repo:latest

```
ubuntu@ip-172-31-33-28:~$ docker push public.ecr.aws/t1t8u9r7/project-repo:latest
The push refers to repository [public.ecr.aws/t1t8u9r7/project-repo]
44136fa355b3: Pushed
e2de96513ba9: Pushed
6711e0c4f5a7: Pushed
f8362b33e9ae: Pushed
3bf3671a4e6a: Pushed
710ec83ca6cd: Pushed
b44956913298: Pushed
70683d58fa5c: Pushed
6638b436592c: Pushed
1fdd0614a47e: Pushed
4f4fb700ef54: Pushed
ea1652ad0ceb: Pushed
latest: digest: sha256:af2b488992b23a4394ad2034da9db826aefcd74ec447818ffbc40ededefed791 size: 856
```

- Verify on ECR using browser



Set Up Amazon EKS and Verify Cluster

» Kubernetes (EKS Cluster)

- Setup Kubernetes in AWS EKS and Confirm EKS cluster is running.
- Write deployment and service YAML files.
- Deploy using kubectl via CodeBuild or Codepipeline.

Amazon EKS Cluster Configuration and Verify Steps

» Project_Steps_4: Kubernetes (EKS Cluster)

- Prerequisites for setup EKS Cluster
- AWS CLI installed and configured (aws configure) with an IAM user that has administrative privileges.

$ aws --version

```
ubuntu@ip-172-31-33-138:~$ aws --version
aws-cli/2.31.35 Python/3.14.4 Linux/7.0.0-1006-aws source/x86_64.ubuntu.26
```

- Verify the AWS CLI configuration

$ aws configure list

```
ubuntu@ip-172-31-33-138:~$ aws configure list
NAME       : VALUE                    : TYPE                 : LOCATION
profile    : <not set>                : None                 : None
access_key : ****************RQM4     : shared-credentials-file :
secret_key : ****************xAfg     : shared-credentials-file :
region     : ap-southeast-2           : config-file          : ~/.aws/config
```

- Download kubectl, Install and Configure kubectl:

$ kubectl version --client

```
ubuntu@ip-172-31-33-138:~$ kubectl version --client
Client Version: v1.37.1
Kustomize Version: v5.8.1
```

- Download and install eksctl:

$ cksctl version

```
ubuntu@ip-172-31-33-138:~$ eksctl version
0.231.0
```

- Create IAM role and attached policy:

$ cat <<EOF > eks-cluster-trust.json

```
{
 "Version": "2012-10-17",
 "Statement": [
  {
```

```
    "Effect": "Allow",
    "Principal": { "Service": "eks.amazonaws.com" },
    "Action": "sts:AssumeRole"
   }
 ]
}
EOF
```

$ aws iam create-role --role-name EksClusterRole --assume-role-policy-document file://eks-cluster-trust.json

```
ubuntu@ip-172-31-33-138:~$ aws iam create-role --role-name EKSClusterRole --assume-role-policy-document file://eks-cluster-trust-policy.json
{
    "Role": {
        "Path": "/",
        "RoleName": "EKSClusterRole",
        "RoleId": "AROAX4POPEIMPV7KXFIG4",
        "Arn": "arn:aws:iam::542202864152:role/EKSClusterRole",
        "CreateDate": "2026-10-20T17:56:56+00:00",
        "AssumeRolePolicyDocument": {
            "Version": "2012-10-17",
            "Statement": [
                {
                    "Effect": "Allow",
                    "Principal": {
                        "Service": "eks.amazonaws.com"
                    },
                    "Action": "sts:AssumeRole"
                }
            ]
        }
    }
}
```

$ aws iam attach-role-policy --role-name EKSClusterRole --policy-arn arn:aws:iam::aws:policy/AmazonEKSClusterPolicy

```
ubuntu@ip-172-31-33-138:~$ aws iam attach-role-policy --role-name EKSClusterRole --policy-arn arn:aws:iam::aws:policy/AmazonEKSClusterPolicy
ubuntu@ip-172-31-33-138:~$ aws iam get-role --role-name EKSClusterRole --query 'Role.Arn' --output text
arn:aws:iam::542202864152:role/EKSClusterRole
```

$ aws iam attach-role-policy --role-name EksClusterRole --policy-arn arn:aws:iam::aws:policy/AmazonEKSClusterPolicy

- Node Group Role and attache role policies:

```
$ cat <<EOF > eks-node-trust.json
{
 "Version": "2012-10-17",
 "Statement": [
  {
   "Effect": "Allow",
   "Principal": { "Service": "ec2.amazonaws.com" },
   "Action": "sts:AssumeRole"
  }
 ]
}
EOF
```

$ aws iam create-role --role-name EksNodeRole --assume-role-policy-document file://eks-node-trust.json

```
ubuntu@ip-172-31-33-138:~$ aws iam create-role --role-name EKSNodeRole --assume-role-policy-document file://node-trust-policy.json
{
    "Role": {
        "Path": "/",
        "RoleName": "EKSNodeRole",
        "RoleId": "AROAX4POPEIMGYVPLN2S2",
        "Arn": "arn:aws:iam::542202864152:role/EKSNodeRole",
        "CreateDate": "2026-10-20T17:59:22+00:00",
        "AssumeRolePolicyDocument": {
            "Version": "2012-10-17",
            "Statement": [
                {
                    "Effect": "Allow",
                    "Principal": {
                        "Service": "ec2.amazonaws.com"
                    },
                    "Action": "sts:AssumeRole"
                }
            ]
        }
    }
}
```

$ aws iam attach-role-policy --role-name myEKSNodeRole --policy-arn arn:aws:iam::aws:policy/AmazonEKSWorkerNodePolicy

$ aws iam attach-role-policy --role-name myEKSNodeRole --policy-arn arn:aws:iam::aws:policy/AmazonEC2ContainerRegistryReadOnly

$ aws iam attach-role-policy --role-name myEKSNodeRole --policy-arn arn:aws:iam::aws:policy/AmazonEKS_CNI_Policy

```
ubuntu@ip-172-31-33-138:~$ aws iam attach-role-policy --role-name EKSNodeRole --policy-arn arn:aws:iam::aws:policy/AmazonEKSWorkerNodePolicy

aws iam attach-role-policy --role-name EKSNodeRole --policy-arn arn:aws:iam::aws:policy/AmazonEC2ContainerRegistryReadOnly

aws iam attach-role-policy --role-name EKSNodeRole --policy-arn arn:aws:iam::aws:policy/AmazonEKS_CNI_Policy
```

- Run the following command to deploy Kubernetes (EKS Managed Cluster)

  $ eksctl create cluster --name MyEksCluster --region ap-southeast-2 --nodegroup-name standard-workers --node-type t3.medium --nodes 2 --nodes-min 1 --nodes-max 2 –managed

or

$ eksctl create cluster -f cluster.yaml; kubectl apply -f workload.yaml

```
ubuntu@ip-172-31-33-138:~$ eksctl create cluster -f cluster.yaml; kubectl apply -f workload.yaml
2026-10-20 15:56:17 [ℹ]  eksctl version 0.231.0
2026-10-20 15:56:17 [ℹ]  using region ap-southeast-2
2026-10-20 15:56:17 [ℹ]  setting availability zones to [ap-southeast-2b ap-southeast-2a ap-southeast-2c]
2026-10-20 15:56:17 [ℹ]  subnets for ap-southeast-2b - public:192.168.0.0/19 private:192.168.96.0/19
2026-10-20 15:56:17 [ℹ]  subnets for ap-southeast-2a - public:192.168.32.0/19 private:192.168.128.0/19
2026-10-20 15:56:17 [ℹ]  subnets for ap-southeast-2c - public:192.168.64.0/19 private:192.168.160.0/19
2026-10-20 15:56:17 [ℹ]  nodegroup "std-workers" will use "" [AmazonLinux2023/1.36]
```

- Verify EKS Cluster (Provision an EKS cluster and ensure it is accessible).

$ eksctl get cluster

```
ubuntu@ip-172-31-33-138:~$ eksctl get cluster
NAME                    REGION          EKSCTL CREATED
EksCluster-1            ap-southeast-2  True
```

$ aws eks list-clusters --region ap-southeast-2

```
ubuntu@ip-172-31-33-138:~$ aws eks list-clusters --region ap-southeast-2
{
    "clusters": [
        "EksCluster-1"
    ]
}
```

$ aws eks update-kubeconfig --region ap-southeast-2 --name EksCluster-1

```
ubuntu@ip-172-31-33-138:~$ aws eks update-kubeconfig --region ap-southeast-2 --name EksCluster-1
Added new context arn:aws:eks:ap-southeast-2:542202864152:cluster/EksCluster-1 to /home/ubuntu/.kube/config
```

$ aws eks describe-cluster --name EksCluster-1 --region ap-southeast-2 --query "cluster.status"

```
ubuntu@ip-172-31-33-138:~$ aws eks describe-cluster --name EksCluster-1 --region ap-southeast-2 --query "cluster.status"
"ACTIVE"
```

- Check core EKS system pods (coredns, aws-node, kube-proxy)

$ eksctl get pods -n kube-system

```
ubuntu@ip-172-31-33-28:~$ kubectl get pods -n kube-system
NAME                       READY   STATUS    RESTARTS   AGE
coredns-69748dc6cc-jnqp2   0/1     Pending   0          28m
coredns-69748dc6cc-sqf8q   0/1     Pending   0          28m
```

- Verify nodes and NodeGroup are in Ready status

$ eksctl get nodegroup --cluster EksCluster-1 --region ap-southeast-2

```
ubuntu@ip-172-31-33-138:~$ eksctl get nodegroup --cluster EksCluster-1 --region ap-southeast-2
CLUSTER         NODEGROUP       STATUS          CREATED                 MIN SIZE        MAX SIZE        DESIRED CAPACITY        INSTANCE TYPE   IMAGE ID                ASG NAME        TYPE
EksCluster-1    NodeGroup       CREATING        2026-10-20T18:25:37Z    1               2               2                       t3.medium       AL2023_x86_64_STANDARD                  managed
```

» Deployment and Service Manifests

- Create a deployment file and apply

$ kubectl apply -f deployment.yml

```
ubuntu@ip-172-31-33-138:~$ cat > deployment.yml <<EOF
apiVersion: apps/v1
kind: Deployment
metadata:
  name: nginx-app
spec:
  replicas: 2
  selector:
    matchLabels:
      app: nginx
  template:
    metadata:
      labels:
        app: nginx
    spec:
      containers:
      - name: nginx
        image: nginx:latest
        ports:
        - containerPort: 80
EOF
ubuntu@ip-172-31-33-138:~$ kubectl apply -f deployment.yml
deployment.apps/nginx-app created
```

$ kubectl get deployments; kubectl get pods

```
ubuntu@ip-172-31-33-138:~$ kubectl get deployments
NAME        READY   UP-TO-DATE   AVAILABLE   AGE
nginx-app   0/2     2            0           41s
ubuntu@ip-172-31-33-138:~$ kubectl get pods
NAME                         READY   STATUS    RESTARTS   AGE
nginx-app-59c4c87bc6-fdkwf   0/1     Pending   0          6m12s
```

- Create service file and apply

$ kubectl apply -f nginx-service.yaml

```
ubuntu@ip-172-31-33-138:~$ cat > service.yaml << EOF
apiVersion: v1
kind: Service
metadata:
  name: nginx-service

spec:
  selector:
    app: nginx

  ports:
  - port: 80
    targetPort: 80

  type: LoadBalancer
EOF
```

$ kubectl apply -f service.yaml

```
ubuntu@ip-172-31-33-138:~$ kubectl apply -f service.yaml
service/nginx-service created
```

```
ubuntu@ip-172-31-33-138:~$ kubectl get svc
NAME            TYPE           CLUSTER-IP     EXTERNAL-IP                                                                PORT(S)        AGE
kubernetes      ClusterIP      10.100.0.1     <none>                                                                     443/TCP        34m
nginx-service   LoadBalancer   10.100.245.7   a0efd57cee5024c769177d0030739599-1443513970.ap-southeast-2.elb.amazonaws.com   80:31154/TCP   5m30s
```

```
ubuntu@ip-172-31-33-138:~$ kubectl get svc nginx-service
NAME            TYPE           CLUSTER-IP     EXTERNAL-IP                                                                PORT(S)        AGE
nginx-service   LoadBalancer   10.100.245.7   a0efd57cee5024c769177d0030739599-1443513970.ap-southeast-2.elb.amazonaws.com   80:31154/TCP   6m38s
```

$ kubectl get nodes

```
ubuntu@ip-172-31-33-138:~$ kubectl get nodes
NAME                                              STATUS   ROLES    AGE   VERSION
ip-172-31-33-192.ap-southeast-2.compute.internal  Ready    <none>   11m   v1.36.4-eks-b9510e3
ip-172-31-4-204.ap-southeast-2.compute.internal   Ready    <none>   11m   v1.36.4-eks-b9510e3
```

$ kubectl get pods -o wide

```
ubuntu@ip-172-31-33-138:~$ kubectl get pods -o wide
NAME                     READY   STATUS    RESTARTS   AGE   IP            NODE                                              NOMINATED NODE   READINESS GATES
nginx-app-59c4c87bc6-fdkwf   1/1     Running   0          89m   172.31.36.68   ip-172-31-33-192.ap-southeast-2.compute.internal   <none>           <none>
nginx-app-59c4c87bc6-qlt8g   0/1     Pending   0          89m   <none>         <none>                                            <none>           <none>
nginx-test-c8697b5cc-5tfm9   1/1     Running   0          15m   172.31.32.51   ip-172-31-33-192.ap-southeast-2.compute.internal   <none>           <none>
```

$ Kubectl get pods -A

```
ubuntu@ip-172-31-33-138:~$ kubectl get pods -A
NAMESPACE     NAME                                 READY   STATUS    RESTARTS   AGE
default       nginx-app-59c4c87bc6-fdkwf           1/1     Running   0          88m
default       nginx-app-59c4c87bc6-qlt8g           0/1     Pending   0          88m
default       nginx-test-c8697b5cc-5tfm9           1/1     Running   0          14m
kube-system   aws-node-c62bh                       2/2     Running   0          3m30s
kube-system   aws-node-s4mr2                       2/2     Running   0          3m31s
kube-system   coredns-69748dc6cc-2lt82             1/1     Running   0          106m
kube-system   coredns-69748dc6cc-flsbb             1/1     Running   0          106m
kube-system   kube-proxy-bfks6                     1/1     Running   0          3m30s
kube-system   kube-proxy-zd95w                     1/1     Running   0          3m31s
```

$ Kubectl get endpoint nginx-service

```
ubuntu@ip-172-31-33-138:~$ kubectl get endpoints nginx-service
Warning: v1 Endpoints is deprecated in v1.33+; use discovery.k8s.io/v1 EndpointSlice
NAME            ENDPOINTS   AGE
nginx-service   <none>      11m
```

```
ubuntu@ip-172-31-33-138:~$ kubectl get all
NAME                 TYPE        CLUSTER-IP    EXTERNAL-IP   PORT(S)   AGE
service/kubernetes   ClusterIP   10.100.0.1    <none>        443/TCP   11s
```

$ kubectl get all

```
ubuntu@ip-172-31-33-138:~$ kubectl get all
NAME                             READY   STATUS    RESTARTS   AGE
pod/nginx-app-59c4c87bc6-fdkwf   1/1     Running   0          101m
pod/nginx-app-59c4c87bc6-qlt8g   0/1     Pending   0          101m
pod/nginx-test-c8697b5cc-5tfm9   1/1     Running   0          26m

NAME                    TYPE           CLUSTER-IP     EXTERNAL-IP                                                                PORT(S)        AGE
service/kubernetes      ClusterIP      10.100.0.1     <none>                                                                     443/TCP        121m
service/nginx-service   LoadBalancer   10.100.245.7   a0efd57cee5024c769177d0030739599-1443513970.ap-southeast-2.elb.amazonaws.com   80:31154/TCP   92m

NAME                        READY   UP-TO-DATE   AVAILABLE   AGE
deployment.apps/nginx-app   1/2     2            1           101m
deployment.apps/nginx-test  1/1     1            1           26m

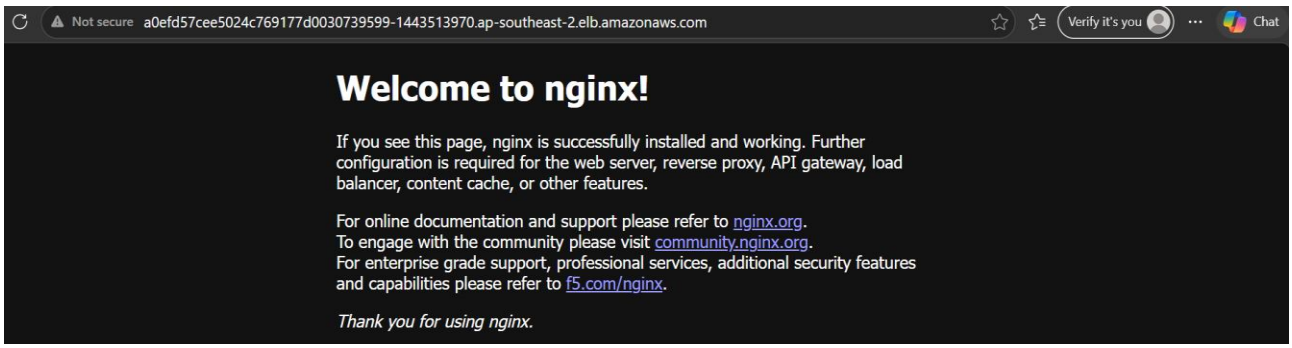
NAME                                   DESIRED   CURRENT   READY   AGE
replicaset.apps/nginx-app-59c4c87bc6   2         2         1       101m
replicaset.apps/nginx-test-c8697b5cc   1         1         1       26m
```

- Verify the application using url with cli and browser as well

```
ubuntu@ip-172-31-33-138:~$ curl http://a0efd57cee5024c769177d0030739599-1443513970.ap-southeast-2.elb.amazonaws.com
<!DOCTYPE html>
<html>
<head>
<title>Welcome to nginx!</title>
<style>
html { color-scheme: light dark; }
body { width: 35em; margin: 0 auto;
font-family: Tahoma, Verdana, Arial, sans-serif; }
</style>
</head>
<body>
<h1>Welcome to nginx!</h1>
<p>If you see this page, nginx is successfully installed and working.
Further configuration is required for the web server, reverse proxy,
API gateway, load balancer, content cache, or other features.</p>

<p>For online documentation and support please refer to
<a href="https://nginx.org/">nginx.org</a>.<br/>
To engage with the community please visit
<a href="https://community.nginx.org/">community.nginx.org</a>.<br/>
For enterprise grade support, professional services, additional
security features and capabilities please refer to
<a href="https://f5.com/nginx">f5.com/nginx</a>.</p>

<p><em>Thank you for using nginx.</em></p>
</body>
</html>
```



Set Up AWS CodeBuild Project

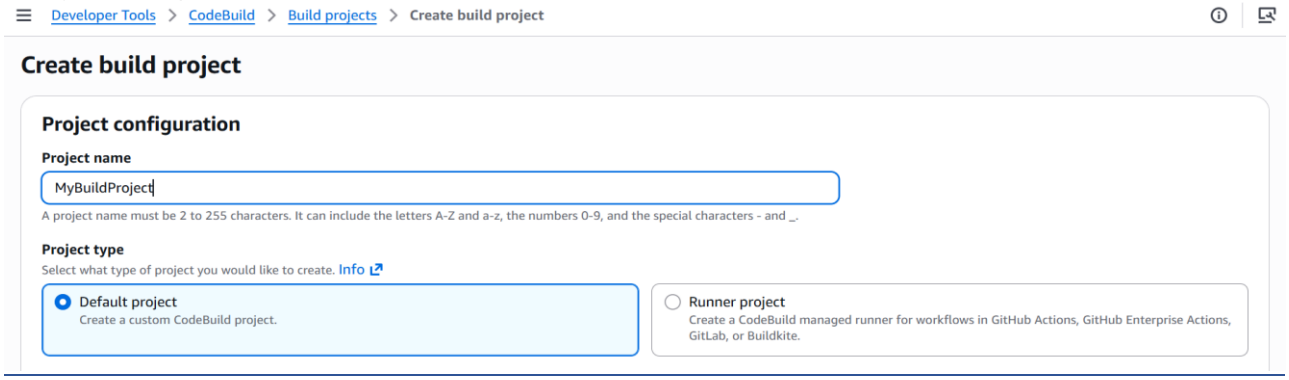
» Create a CodeBuild project:

- Source: Connect to your repository
- Environment: Use managed image (Amazon Linux, Ubuntu)
- Write and define commands in buildspec.yml.

Steps to Create Docker Hub Repository:

» Project_Steps_5: Create a CodeBuild project:

- Open the AWS Management Console, Navigate to CodeBuild and Click Create build project.
- Project Configuration:



» Environment Configuration: Select Environment image: Managed image (Operating system: Ubuntu Linux)

- Image: aws/codebuild/standard:latest

» Service Role (New service role): codebuild-sample-service-role

» Buildspec Configuration: Select: Use a buildspec file CodeBuild

- buildspec.yml (Node.js)

```
$ cat buildspec.yml
version: 0.2
phases:
  install:
    runtime-versions:
      nodejs: 20
    commands:
      - echo Installing dependencies
      - npm install
```

```
  pre_build:
    commands:
      - echo Running lint checks
      - npm run lint
  build:
    commands:
      - echo Building application
      - npm run build
  post_build:
    commands:
      - echo Build completed successfully
artifacts:
  files:
    - '**/*'

$ cat buildspec.yml (Node.js)
version: 0.2
phases:
  install:
    runtime-versions:
      nodejs: 20
    commands:
      - echo Installing dependencies
      - npm install
  pre_build:
    commands:
      - echo Running lint checks
      - npm run lint
  build:
    commands:
      - echo Building application
      - npm run build
  post_build:
    commands:
      - echo Build completed successfully
artifacts:
  files:
    - '**/*'

$ cat buildspec.yml (Java Maven)
version: 0.2
phases:
  install:
    commands:
      - echo Installing Maven dependencies
  build:
    commands:
      - mvn clean package
```

```
artifacts:
  files:
    - target/*.jar

$ cat buildspec.yml (Docker)
version: 0.2
phases:
  pre_build:
    commands:
      - aws ecr get-login-password --region ap-southeast-2 | docker login --username AWS --password-
stdin <ACCOUNT_ID>.dkr.ecr.ap-southeast-2.amazonaws.com
  build:
    commands:
      - docker build -t sample-app .
      - docker tag sample-app:latest <ACCOUNT_ID>.dkr.ecr.ap-southeast-2.amazonaws.com/sample-
app:latest
  post_build:
    commands:
      - docker push <ACCOUNT_ID>.dkr.ecr.ap-southeast-2.amazonaws.com/sample-app:latest
artifacts:
  files:
    - '**/*'
```

» Enable Build Triggers (Optional)
- Under **Build triggers**: Enable webhook
- Trigger on: Push / Pull Request (Branch: main, Event: PUSH) Every git push automatically starts a CodeBuild job.

» Create Project (Create Build Project):
-

» Start a Build (From the project page): Build Project → Start Build
- Build History
- CloudWatch Logs
- Build Artifacts

» CLI Alternative (Create a project using AWS CLI):

```
$ aws codebuild create-project \
--name sample-codebuild-project \
--source type=GITHUB,location=https://github.com/<username>/<repo>.git \
--artifacts type=NO_ARTIFACTS \
--environment type=LINUX_CONTAINER,\
image=aws/codebuild/amazonlinux-x86_64-standard:5.0,\
computeType=BUILD_GENERAL1_SMALL,\
privilegedMode=true \
 --service-role arn:aws:iam::<ACCOUNT_ID>:role/codebuild-service-role
```

- Verify:

```
$ aws codebuild list-projects
$ aws codebuild batch-get-projects --names sample-codebuild-project
```

Thanks